Deployment Guide

Project: Enterprise Quantum Computing Platform — QFT Bank **Student:** Syed Muhammad Tahoor Ali **Diploma:** EduQual Level 6, Diploma in Artificial Intelligence Operations (ANPP-OP)

---

1. Purpose

The platform supports three deployment paths, in increasing order of production realism: Docker Compose (fastest local full stack), Kubernetes on Minikube (the assessed deployment target), and Terraform (the same platform declared as Infrastructure as Code). This guide covers all three, assuming the prerequisites in the Installation Guide are in place.

---

2. Path A — Docker Compose (Local Full Stack)

The quickest way to run everything — database, cache, and backend — together.

```
cp .env.example .env        # if not already done
docker compose up -d
docker compose logs -f backend
```

Compose starts three services defined in `docker-compose.yml`: PostgreSQL 16 (Alpine) with a named volume for data persistence and a `pg_isready` health check; Redis 7 (Alpine) with a `redis-cli ping` health check; and the backend, built from `backend/Dockerfile`, which waits for both datastores to report healthy before starting (`depends_on: condition: service_healthy`).

Confirm health:

```
curl http://localhost:8000/health
```

Because Compose brings the database up healthy before the backend starts, the expected response reports `"storage_backend":"postgresql"`. Swagger UI is at `http://localhost:8000/docs`.

The frontend is not part of the Compose file; for a full UI experience use the Kubernetes path, or run the frontend in Vite dev mode against the Compose backend.

Tear down with `docker compose down` (add `-v` to also remove the database volume).

---

3. Path B — Kubernetes (Minikube) — Assessed Target

This is the deployment demonstrated in the assessment. The cluster runs eight workloads, of which only the frontend and the two monitoring UIs are externally reachable.

3.1 Start Minikube and point Docker at it

Images must be built into Minikube's own Docker daemon, because the manifests use `imagePullPolicy: Never` — there is no external registry.

```
minikube start
eval $(minikube docker-env)
```

3.2 Build the application images

```
docker build -t qft-backend:latest ./backend
docker build -t qft-frontend:latest ./frontend
```

The backend image is multi-layer Python 3.13-slim and runs as a non-root user (UID 1000) with a hardened `securityContext`. The frontend is a two-stage build: Node compiles the React production bundle, then only the compiled static files are copied into a small nginx image that serves the SPA and reverse-proxies `/api` to the backend.

3.3 Apply the manifests

```
kubectl apply -f infra/k8s/
```

This creates, in `infra/k8s/` order: PostgreSQL (with a PersistentVolumeClaim), Redis, the backend, the frontend, Prometheus, Grafana, Loki, and Promtail. Wait for readiness:

```
kubectl get pods -w
```

3.4 Access points

Service	URL	Notes
Application (frontend)	`http://<minikube-ip>:30080`	The only application entry point
Prometheus	`http://<minikube-ip>:30090`	Metrics UI
Grafana	`http://<minikube-ip>:30300`	Dashboards

Obtain the node IP with `minikube ip`. The backend, PostgreSQL, and Redis are `ClusterIP`-only and are never reachable directly from outside — the browser talks only to nginx, which proxies to the backend over the cluster network.

3.5 Rebuilding after a code change

Because the manifests use `imagePullPolicy: Never`, a code change requires rebuilding the image into Minikube's Docker and restarting the deployment:

```
eval $(minikube docker-env)
docker build -t qft-backend:latest ./backend
kubectl rollout restart deployment/backend
kubectl rollout status deployment/backend
```

The same pattern applies to the frontend.

3.6 Startup ordering note

In Kubernetes, pods start in parallel, so the backend may briefly come up before PostgreSQL is ready. The persistence layer falls back to in-memory storage in that window rather than crashing; a `kubectl rollout restart deployment/backend` reconnects it once the database is up. The active backend is always visible via `/health`. This behaviour is intentional resilience; a production deployment would enforce ordering with an init-container or readiness gate.

---

4. Path C — Terraform (Infrastructure as Code)

Terraform deploys the same platform declaratively into a dedicated namespace (`qft-terraform`), deliberately separate from the `kubectl`-managed `default` namespace so the two do not interfere.

Prerequisites: Minikube running, and the images already built into Minikube (as in the Kubernetes path).

```
cd infra/terraform
terraform init          # downloads the Kubernetes provider
terraform plan          # previews every resource to be created
terraform apply         # type 'yes' to deploy
kubectl get pods -n qft-terraform
minikube service frontend -n qft-terraform --url
```

The Terraform frontend is exposed on NodePort `30081` (distinct from the `kubectl` deployment's `30080`), so both can run side by side. Tear down cleanly with:

```
terraform destroy
```

Terraform state files are git-ignored, as they can contain sensitive values.

The value of this path is reproducibility: the entire platform — database, cache, backend, frontend — is version-controlled code provisioned with a single command, previewable before apply and destroyable cleanly. In cloud production, the same workflow would additionally provision the underlying managed cluster (EKS or GKE).

---

5. Choosing a Path

Use Docker Compose for the fastest backend-plus-data local run. Use Kubernetes for the full, monitored, orchestrated deployment as assessed. Use Terraform to demonstrate that the same deployment is fully codified. All three describe the same application; they differ only in how the infrastructure around it is created.